



MCT344: Industrial Robotics Project

Group 14

Name	Id
Ahmed Hossam Eissa	2200138
Ashraf Ahmed Barakat	2200403
Ahmed Sayed Ismael	2200410
Nassir Afifi Nassir	2200697

Submitted to : Dr. Mohamed Ibrahim

Dr. Diaa Emad

Eng. Samy

Eng. Hamdy

Eng. Martina

Table of Contents

1. Robot description and specifications.....	
2. DH parameters and modeling assumptions.....	
3. Full analytical derivations.....	
4. Mathematical equations and explanations.....	
5. Simulation methodology.....	
6. Results and plots.....	
7. Comparative study between methods.....	
8. Drive Link.....	

1. Robot Description and Specifications

1.1 Robot Description

The selected robotic system is a four-degree-of-freedom (4-DOF) articulated robotic manipulator developed by the project team. The robot is designed for pick-and-place operations in industrial environments, such as material handling, packaging, and sorting applications.

The manipulator consists of four revolute joints (RRRR configuration) and an end-effector gripper used for grasping and transporting objects. The gripper is considered an end-effector and is not included in the robot's degrees of freedom.

The robot provides sufficient workspace coverage and flexibility to perform object transfer tasks between predefined locations while maintaining smooth and accurate motion.

1.2 Robot Configuration

- Robot Type: Articulated Manipulator
- Degrees of Freedom (DOF): 4
- Joint Configuration: R-R-R-R
- End Effector: Two-finger Gripper
- Actuation: Servo Motors
- Application: Pick-and-Place Operations



1.3 Robot Specifications

Parameter	Value
Number of Joints	4
Joint Type	Revolute
Degrees of Freedom	4
End Effector	Gripper
Total Weight	695 gm
Motor Type	Servo Motors
Control Method	Joint-Space Position Control

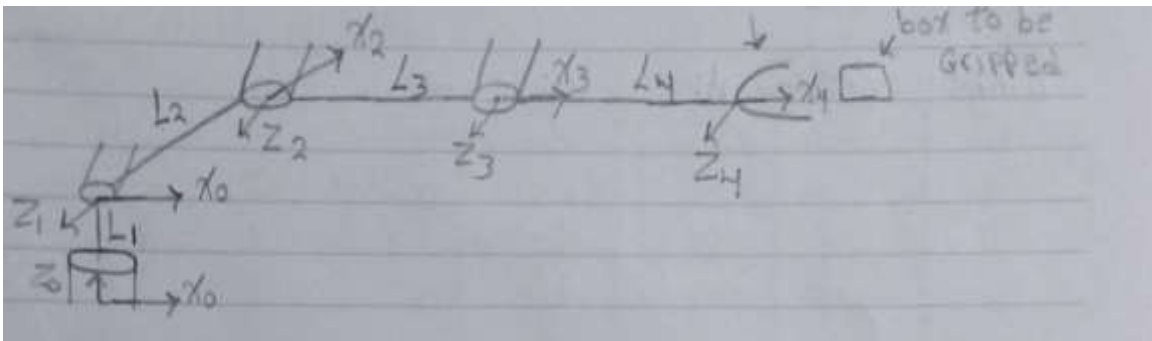
1.4 Link Dimensions

Link	Length (m)
Link 1	0.0564
Link 2	0.17
Link 3	0.15
Link 4	0.08364

The exact dimensions are obtained from the CAD model and are used in the kinematic and dynamic analysis.

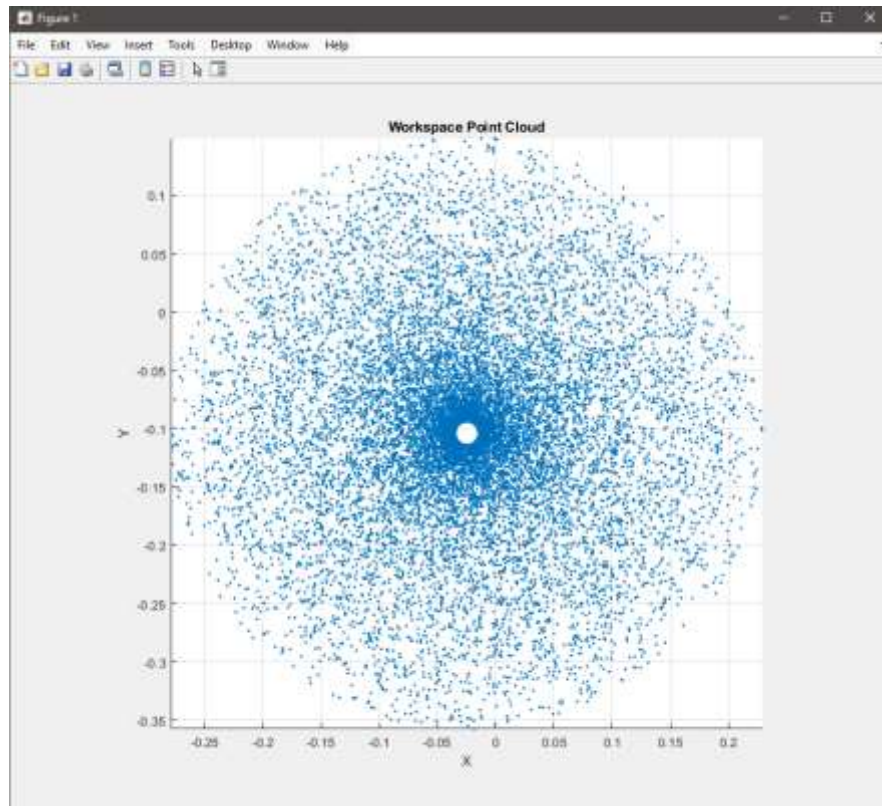
2. DH-Table and modeling assumption

2.1 DH-table

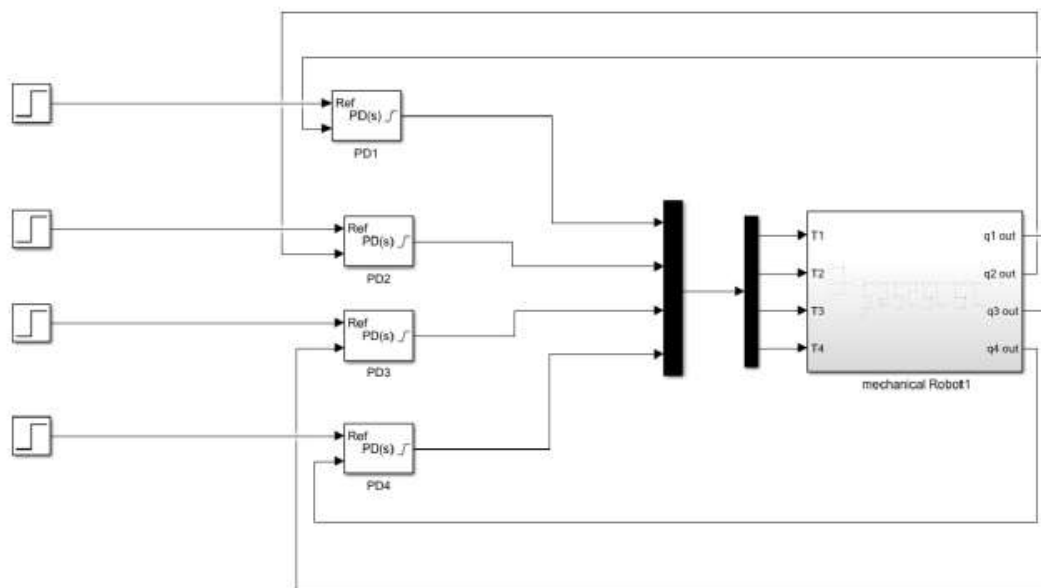


Link	θ_i	d_i	a_i	α_i
1	θ_1	L1	0	90
2	θ_2	0	L2	0
3	θ_3	0	L3	0
4	θ_4	0	L4	0

Robot Workspace :



Robot Control :-



2.2 Modeling assumptions

In order to simplify the mathematical modeling of the robotic manipulator and make the system suitable for simulation in MATLAB, several assumptions are made:

- The robot links are considered rigid bodies, with no deformation during motion.
- All joints are assumed to be ideal revolute joints with no friction, backlash, or mechanical clearance.
- Air resistance and external disturbances are neglected.
- The mass of each link is assumed to be either uniformly distributed or concentrated at its center of mass.
- The gravitational field is constant and acts only in the vertical direction.
- The system is deterministic, meaning no noise or uncertainty is considered in sensors or actuators.
- Actuator dynamics (motor electrical behavior) are simplified or neglected unless otherwise stated.

These assumptions reduce the complexity of the model while preserving the essential dynamic behavior of the robotic arm

3. Full Analytical Derivations

The mathematical model of the robotic arm is derived using standard rigid body dynamics and kinematic relationships.

3.1 Forward Kinematics

The position of the end-effector is determined using transformation matrices based on Denavit–Hartenberg (DH) parameters. Each link transformation is represented as:

The overall transformation from base to end-effector is given by:

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_i \\ \sin \theta_i \cos \alpha_i & \cos \theta_i \cos \alpha_i & -\sin \alpha_i & -d_i \sin \alpha_i \\ \sin \theta_i \sin \alpha_i & \cos \theta_i \sin \alpha_i & \cos \alpha_i & d_i \cos \alpha_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From this matrix, the position of the end-effector is extracted.

Total Transformation matrix :

$$\begin{aligned}
 {}^0T_4 &= {}^0T_1 \times {}^1T_2 \times {}^2T_3 \times {}^3T_4 \\
 &= \begin{bmatrix} C_1 & 0 & S_1 & 0 \\ S_1 & 0 & -C_1 & 0 \\ 0 & 1 & 0 & L_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} C_2 & -S_2 & 0 & L_2 C_2 \\ S_2 & C_2 & 0 & L_2 S_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} C_3 & -S_3 & 0 & L_3 C_3 \\ S_3 & C_3 & 0 & L_3 S_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} C_4 & 0 & -S_4 & L_4 C_4 \\ S_4 & 0 & C_4 & L_4 S_4 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} C_1 [C_{23} C_4 - S_{23} S_4] & -C_1 [C_{23} S_4 + S_{23} C_4] & S_1 & C_1 [L_2 C_2 + L_3 C_{23} + L_4 C_{234}] \\ S_1 [C_{23} C_4 - S_{23} S_4] & -S_1 [C_{23} S_4 + S_{23} C_4] & -C_1 & S_1 [L_2 C_2 + L_3 C_{23} + L_4 C_{234}] \\ S_{23} C_4 + C_{23} S_4 & -[S_{23} S_4 - C_{23} C_4] & 0 & L_1 + L_2 S_2 + L_3 S_{23} + L_4 S_{234} \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

3.2 Dynamic Modeling

The dynamic behavior of the manipulator is derived using the Lagrangian formulation :

$$L = T - V$$

where:

- T is the total kinetic energy
- V is the potential energy

The equations of motion are obtained using :

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}_i} \right) - \frac{\partial L}{\partial \theta_i} = \tau_i$$

This leads to the standard robot dynamics equation :

$$\tau = M(\theta)\ddot{\theta} + C(\theta, \dot{\theta})\dot{\theta} + G(\theta)$$

4. Mathematical equations

End Effector Position:

```
Px = cos(t1)*(L3*cos(t2 + t3) + L2*cos(t2) + L4*cos(t2 + t3 + t4))  
Py = sin(t1)*(L3*cos(t2 + t3) + L2*cos(t2) + L4*cos(t2 + t3 + t4))  
Pz = L1 + L3*sin(t2 + t3) + L2*sin(t2) + L4*sin(t2 + t3 + t4)
```

5. Simulation Methodology

The robotic arm model is implemented and simulated using MATLAB/Simulink.

5.1 Implementation Approach

The derived kinematic and dynamic equations are implemented using MATLAB scripts and/or Simulink blocks. The system is represented in state-space form to allow numerical solution using built-in solvers.

5.2 Numerical Solver

The differential equations are solved using numerical integration methods such as **ode45**, which is based on the Runge-Kutta method.

5.3 Input Signals

The system is tested using predefined inputs such as:

- Step input in joint torque
- Sinusoidal reference trajectories
- Desired joint angle profiles

5.4 Outputs

The simulation provides:

- Joint angles over time
- Joint velocities and accelerations
- End-effector trajectory in Cartesian space

5.5 Validation

The simulation results are validated by:

- Checking physical consistency of motion
- Comparing results with theoretical expectations
- Ensuring energy and motion behavior are realistic

6. Results and Plots

6.1 Method 1

6.1.1 Forward kinematics :

```
End Effector Position (cm)
-15.1197
-26.1835
23.8023
```

6.1.2 Inverse Kinematics :

```
Joint Angles (deg)
59.9159 45.3666 30.4256 -0.7214
```

6.1.3 Jacobian :

```
Geometric Jacobian (6x4):
-0.1305  -0.1829  -0.1253  -0.0404
 0.0753  -0.3169  -0.2171  -0.0700
      0    0.1507   0.0505   0.0216
      0   -0.8660  -0.8660  -0.8660
      0    0.5000   0.5000   0.5000
 1.0000      0      0      0
```

6.1.4 Dynamics

```
Inertia Matrix M(q)
 0.0057  0.0000  -0.0000  0.0000
 0.0000  0.0191  0.0084  0.0015
-0.0000  0.0084  0.0045  0.0009
 0.0000  0.0015  0.0009  0.0002
```

```
Gravity Vector G(q)
 0
 0.3626
 0.0677
 0.0106
```

```
Coriolis Matrix C(q,qdot)
 0  0  0  0
 0  0  0  0
 0  0  0  0
 0  0  0  0
```

```
Joint Torque Vector
 0
 0.3626
 0.0677
 0.0106
```

6.2 Method 2

6.2.1 Forward kinematics

X = -0.1502 m

Y = -0.2599 m

Z = 0.2366 m

Transformation Matrix:

0.1298	-0.4833	0.8658	-0.1502
0.2246	-0.8361	-0.5005	-0.2599
0.9658	0.2595	0.0000	0.2366
0	0	0	1.0000

6.2.2 Inverse kinematics

Joint Angles (deg)

60.0184

43.5659

31.8054

6.7956

6.2.3 Jacobian

```
Full Jacobian
  -0.1565  -0.1729  -0.1128  -0.0404
    0.0903  -0.2995  -0.1954  -0.0700
         0    0.1807   0.0605   0.0216
         0   -0.8660  -0.8660  -0.8660
         0    0.5000   0.5000   0.5000
    1.0000         0         0         0

Rank(J) = 4
Singular Values
  1.7797
  1.0162
  0.2131
  0.0336

Linear Velocity
  -0.1137
  -0.0162
   0.0366

Angular Velocity
  -0.3464
   0.2000
   0.5000

Required Joint Velocities
  -0.2397
   0.0761
  -0.2016
  -0.0722
```

6.2.3 Dynamics

```
Inertia Matrix M(q)
  0.0059    0.0000    0.0000    0.0000
  0.0000    0.0084    0.0034    0.0005
  0.0000    0.0034    0.0018    0.0003
  0.0000    0.0005    0.0003    0.0001
```

```
=====
Coriolis and Centrifugal Vector
```

```
1.0e-03 *
```

```
  0.7139
 -0.9350
 -0.1588
 -0.0258
```

```
=====
Gravity Vector G(q)
```

```
-0.1288
-0.1559
-0.0246
-0.0033
```

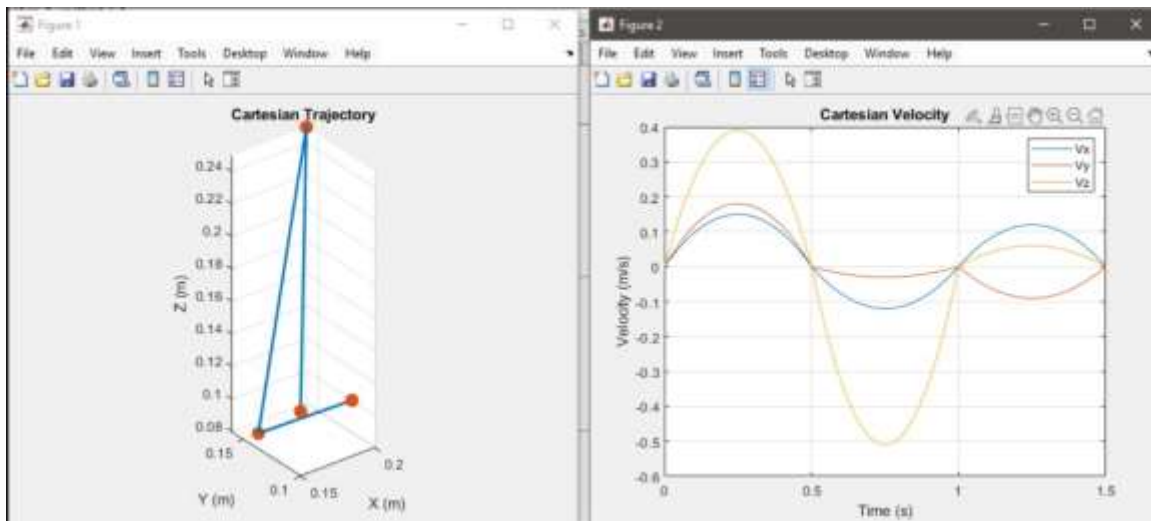
```
=====
Joint Torques (From Equation)
```

```
-0.1221
-0.1568
-0.0247
-0.0033
```

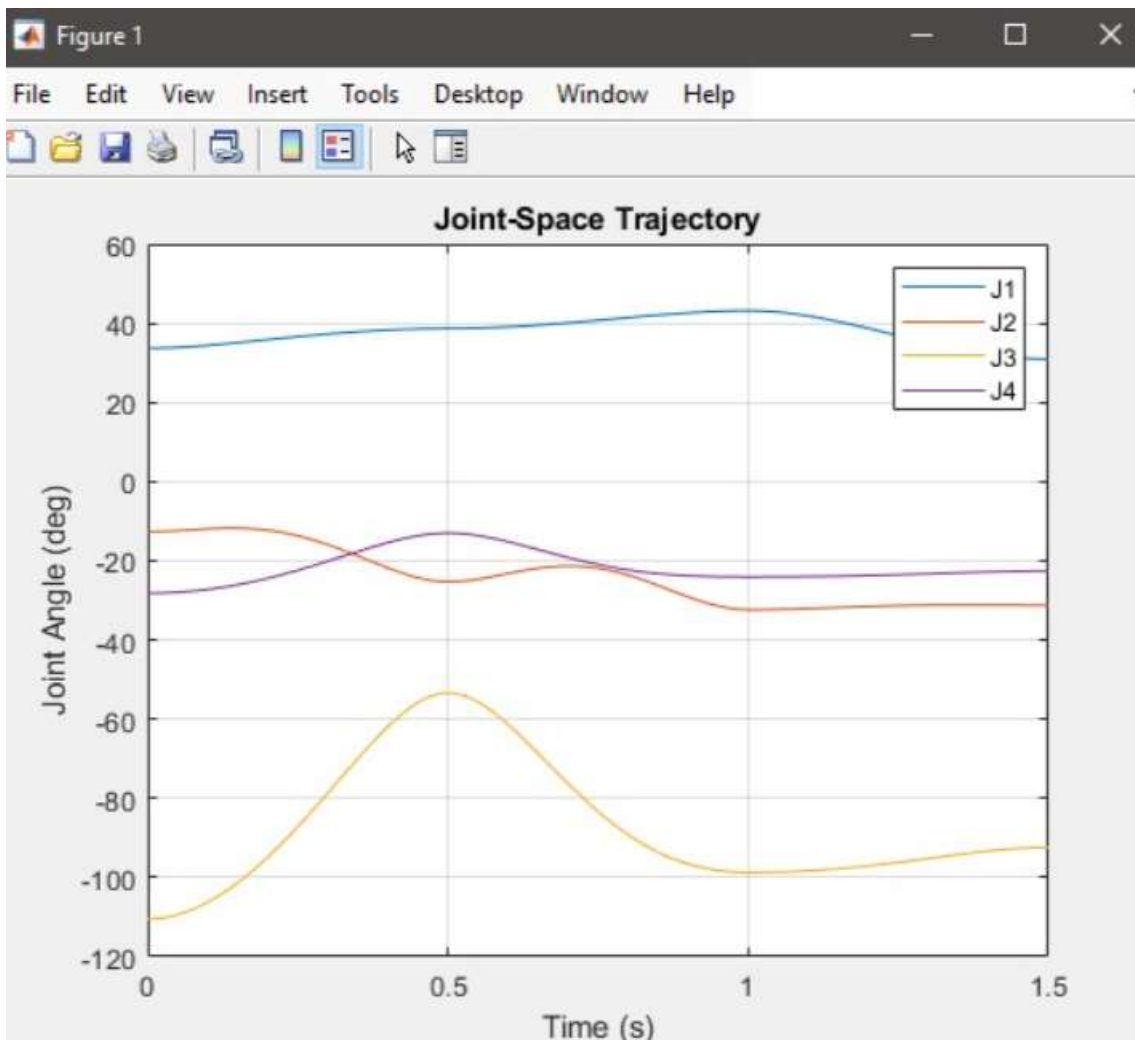
```
=====
Joint Torques (inverseDynamics)
```

```
-0.1221
-0.1568
-0.0247
-0.0033
```

6.2.4 Cartesian Trajectory

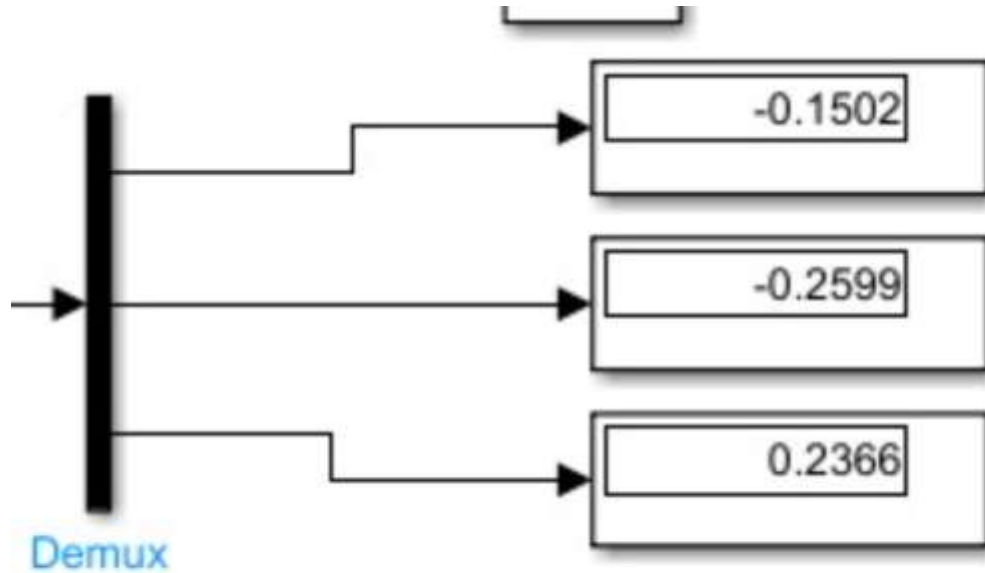


6.2.4 Joint Space

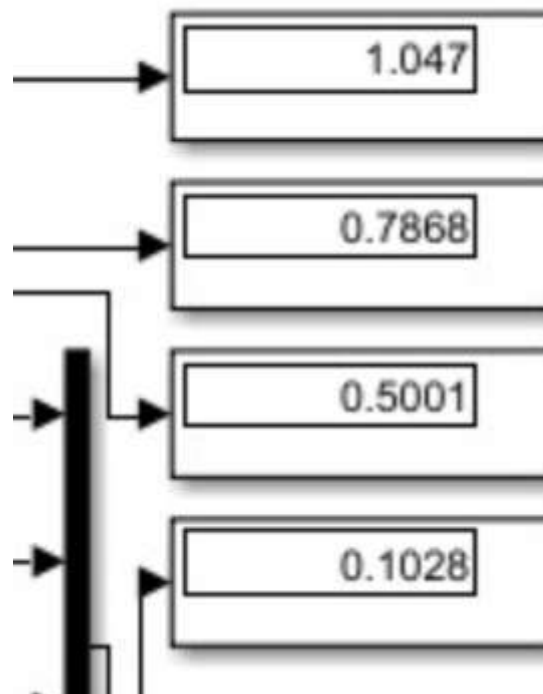


6.3 Method 3

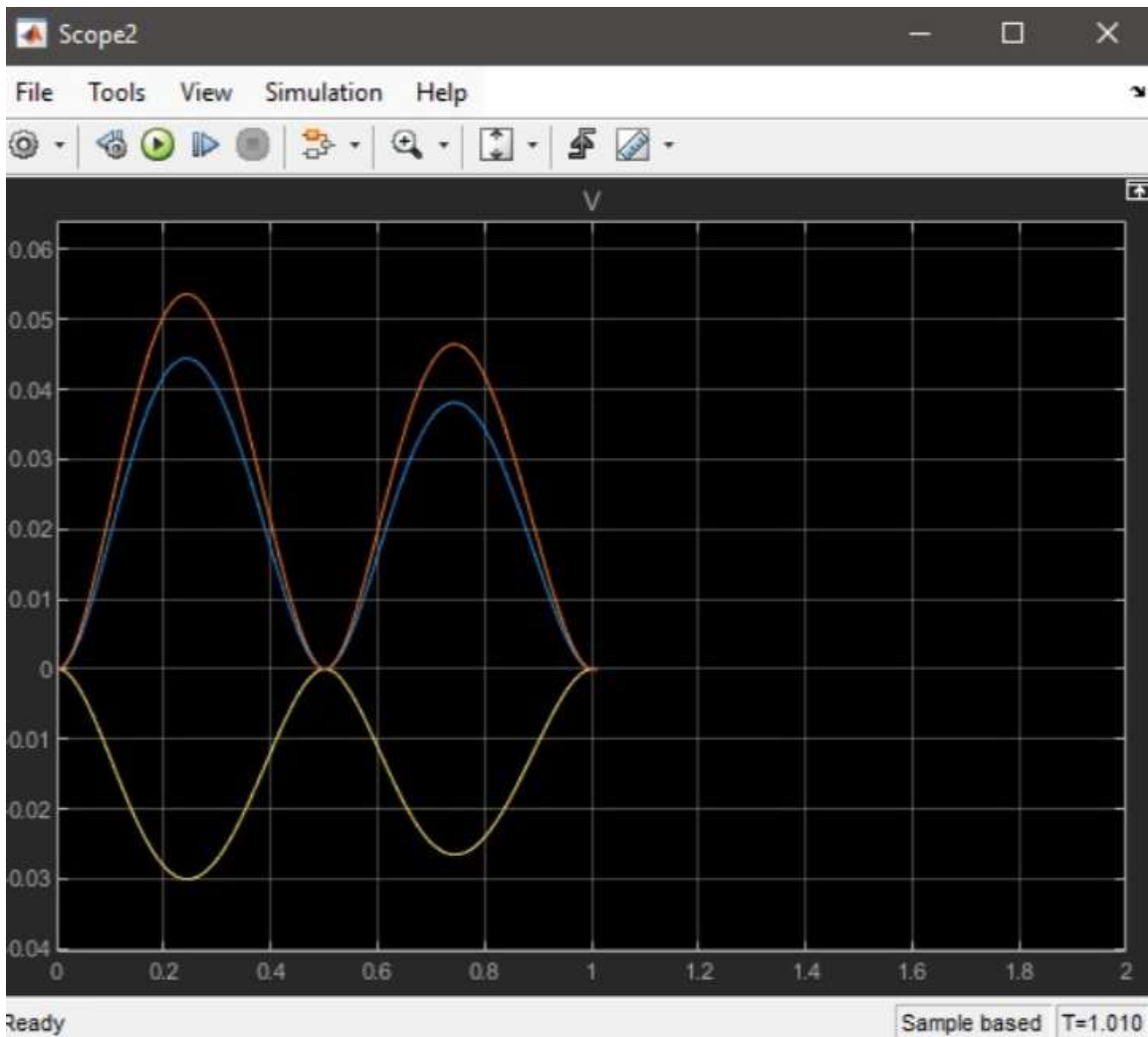
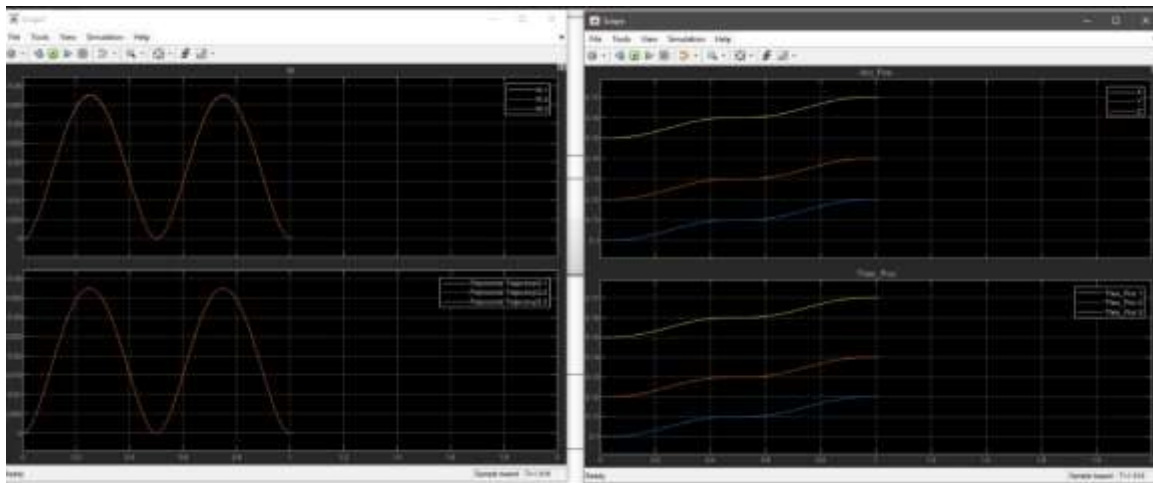
6.3.1 Forward Kinematics



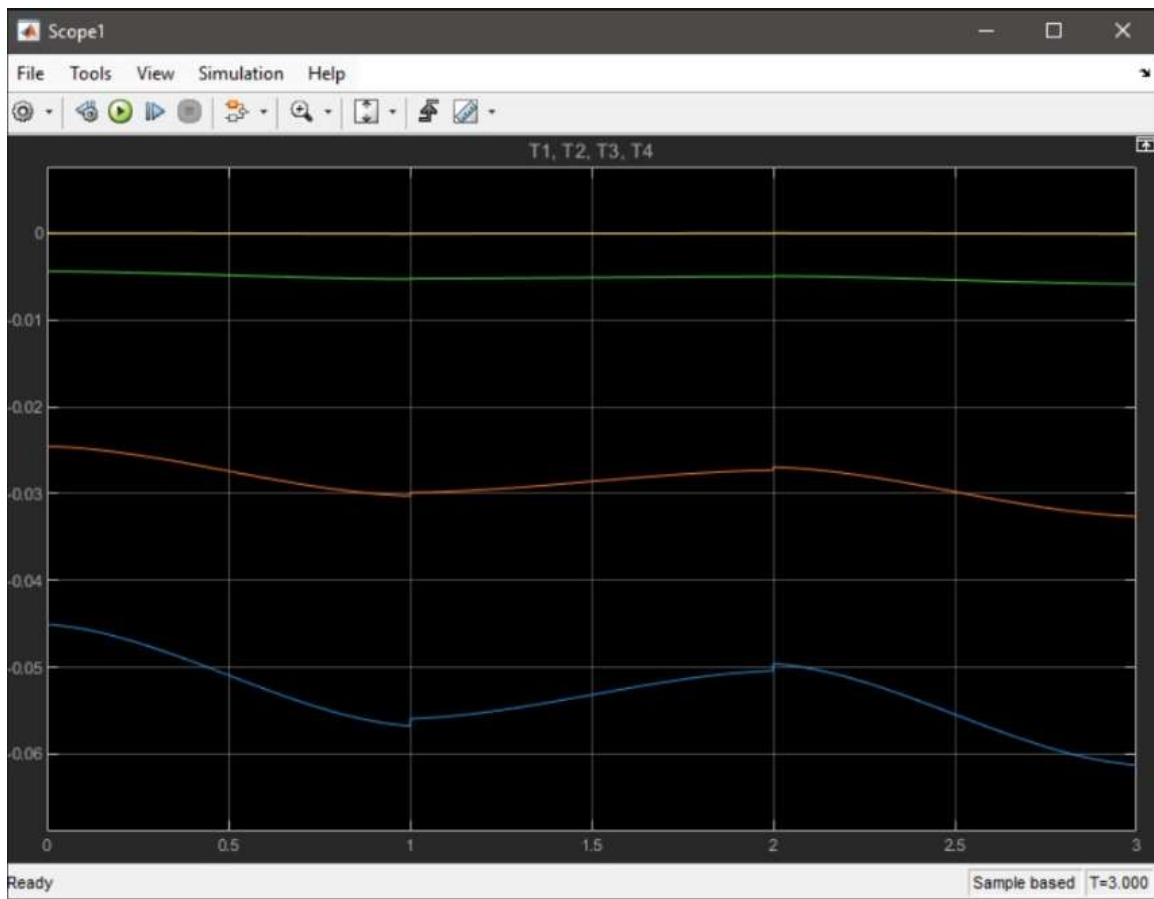
6.3.2 Inverse Kinematics



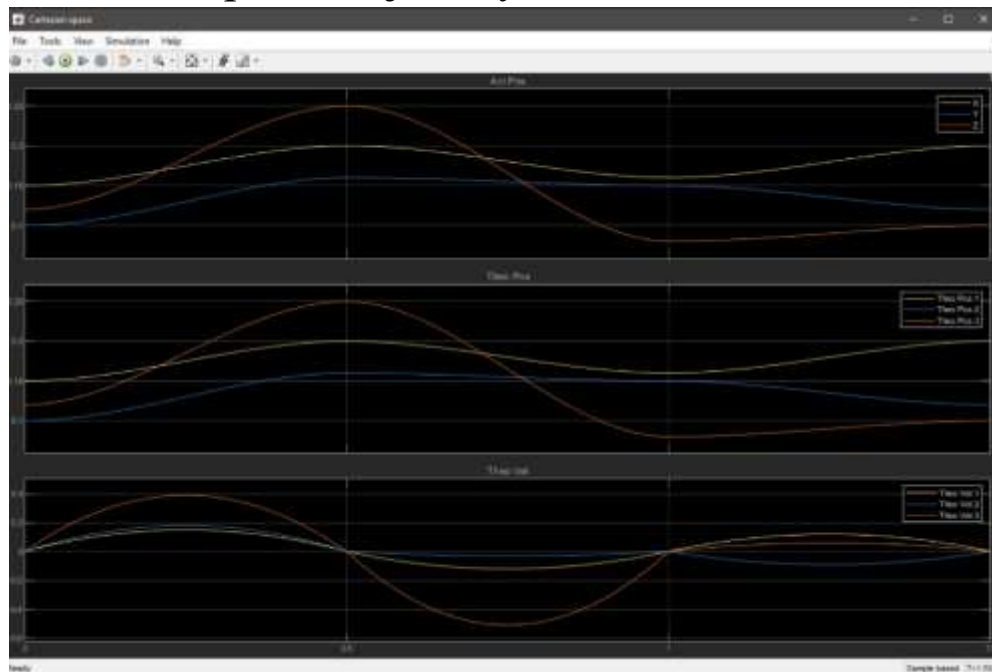
6.3.3 Jacobian



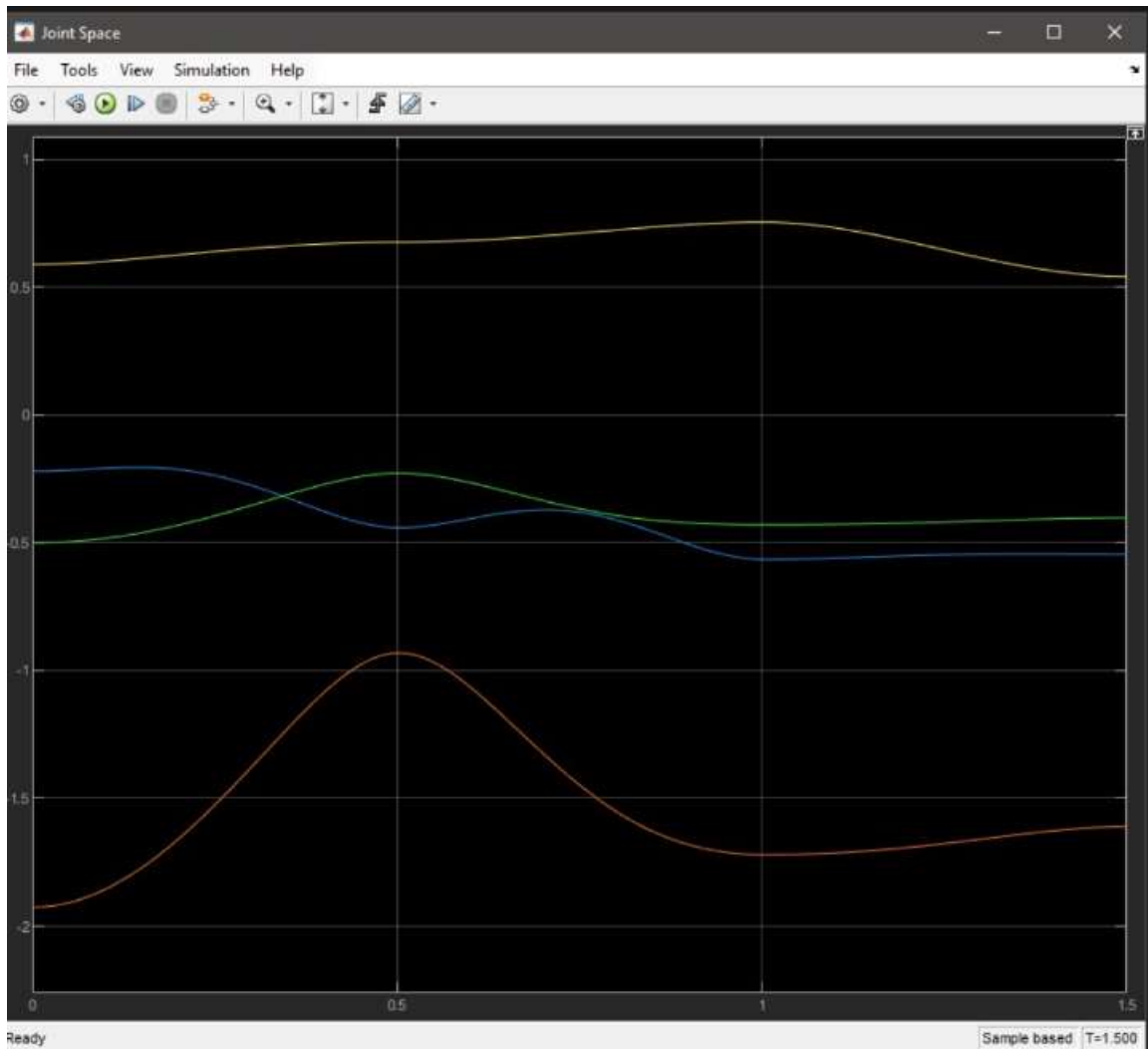
6.3.4 Dynamics



6.3.5 Cartesian Space Trajectory



6.3.6 Joint Space



6.3 Bonus Work: Experimental Validation of Inverse Kinematics and Trajectory Tracking

Methodology

1. A set of desired Cartesian waypoints, defined by the end-effector positions $((x, y, z))$, was specified.
2. An analytical inverse kinematics (IK) algorithm was developed in MATLAB to compute the corresponding joint angles $((q_1, q_2, q_3, q_4))$.
3. A smooth joint-space trajectory was generated using fifth-order polynomial interpolation (Quintic Polynomial Trajectory Planning).
4. The generated joint trajectories were transmitted from MATLAB to the Arduino controller through serial communication.
5. The Arduino board commanded the servomotors through the PCA9685 servo driver module.
6. Encoder feedback from the robot was returned to MATLAB and recorded for performance evaluation.

Experimental Validation

The actual joint angles measured from the robot were compared with the desired trajectories generated in MATLAB. The tracking performance was evaluated using the Root Mean Square Error (RMSE) for each joint.

Furthermore, the forward kinematics equations were employed to reconstruct the end-effector trajectory from both the desired and experimental joint angles. A three-dimensional comparison between the planned and actual TCP (Tool Center Point) paths was then plotted.

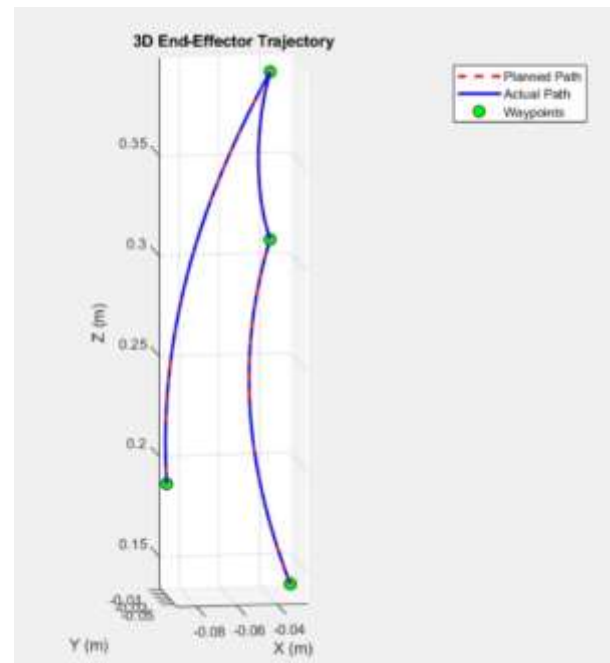
Results

The experimental results demonstrated that:

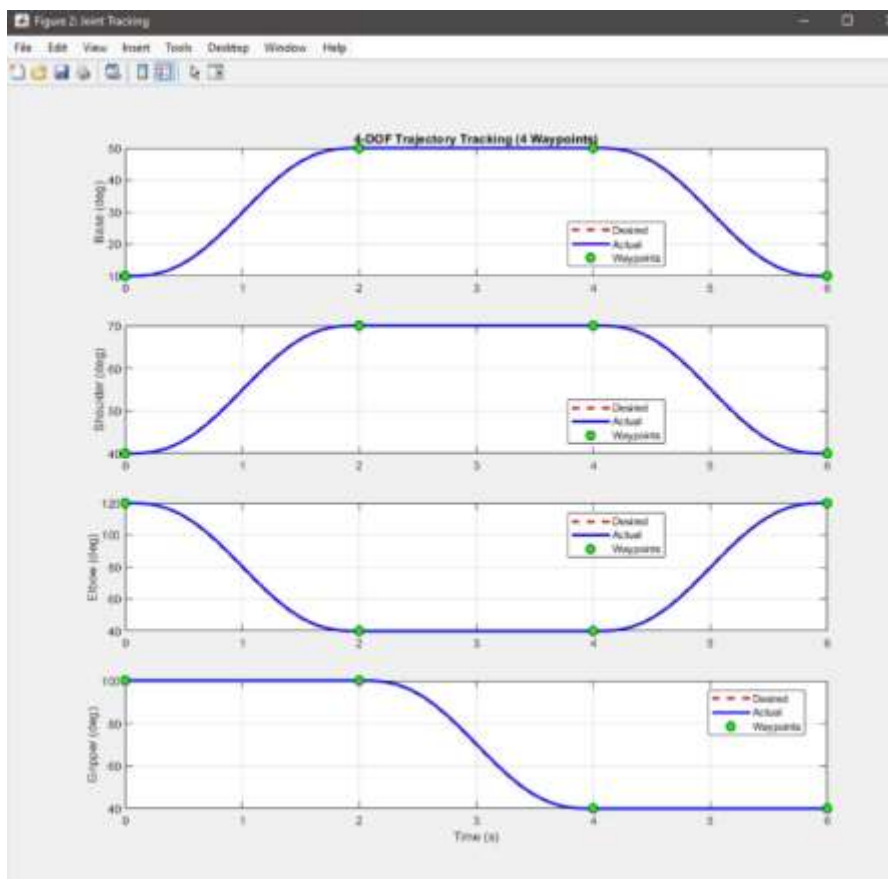
- **The analytical inverse kinematics algorithm successfully generated feasible joint configurations for the desired end-effector positions.**
- **The robot was able to follow the generated trajectories smoothly.**
- **The actual joint responses closely matched the desired trajectories with relatively small tracking errors.**
- **The experimentally reconstructed TCP path exhibited good agreement with the planned path obtained from simulation.**

These results confirm the validity of the proposed analytical inverse kinematics model and demonstrate the effectiveness of the trajectory planning and experimental implementation on the physical robotic manipulator.

6.3.1 Cartesian space trajectory



6.3.2 Joint Space Trajectory



7. Comparative study between methods

Criterion	Method 1 Analytical	Method 2: Robotics Toolbox	Method 3: Simscape / Simulink
Mathematical Effort	Maximum (Manual Derivations)	Minimum (Built-in functions)	None (Physical Blocks)
Execution Speed	Very Fast (Direct algebraic equations)	Fast (Optimized toolbox code)	Slower (Solver-dependent simulation)
Accuracy	Theoretical Exact	Theoretical Exact	Numerical Approximation (Solver errors)
Debugging / Troubleshooting	Hard (Hard to trace math errors)	Easy (Clear errors from functions)	Medium (Visual tracking & solver tuning)

- After completing the project, three different methods were implemented to evaluate the system response. It was observed that Method 1 and Method 2 produce results that are very close to the dynamic analysis, while Method 3 shows slightly larger deviations.

The small differences between Method 1, Method 2, and Method 3 are mainly due to the simplifying assumptions adopted in Method 1 and Method 2, which include:

- Link masses are lumped at simplified center-of-mass locations.
- The center of mass (COM) is assumed to be located near the midpoint of each link.
- The rotational inertia of the links is neglected or significantly simplified, with the dominant kinetic energy assumed to be translational motion only.

8 Drive Link

<https://drive.google.com/drive/folders/12V9QaTVXyAH5NKWv65WDeGFenN2h7tYs3?usp=sharing>